



UNIVERSIDAD CENTRAL DE VENEZUELA · FACULTAD DE CIENCIAS · ESCUELA DE COMPUTACIÓN

Diseño e implementación de un modelo de mantenimiento asistido por inteligencia artificial para una arquitectura de microservicios en el sistema PortalAsig

Trabajo Especial de Grado

AUTOR

Br. Frank Ángel Ponte Delgado

TUTOR

Prof. Walter Hernández

FECHA

Caracas, 2026

Contenido de la presentación

01 Contexto y planteamiento del problema

02 Objetivos del trabajo

03 Marco conceptual (síntesis)

04 Solución: microservicios, frontend y PAIMA

05 Validación y resultados

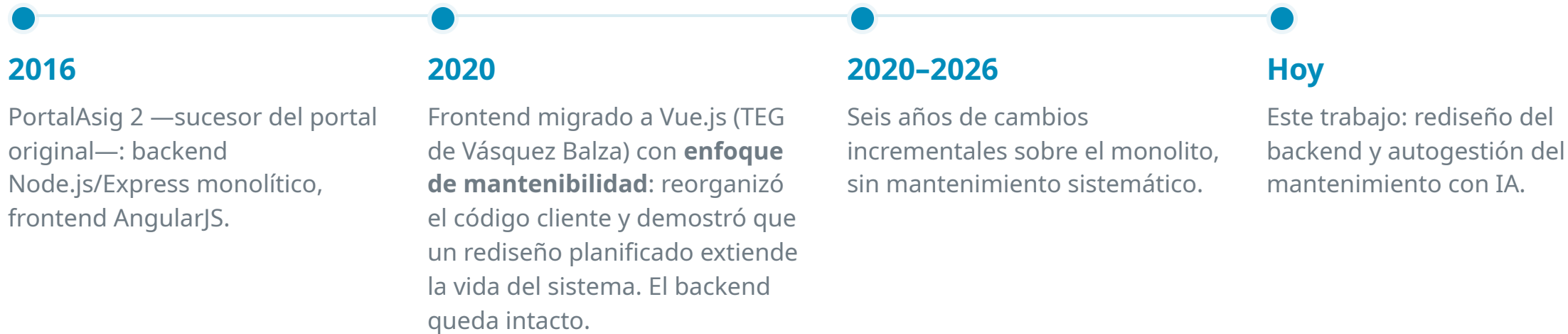
06 Conclusiones, limitaciones y trabajo futuro

SECCIÓN

01

Contexto y problema

PortalAsig: una década de evolución



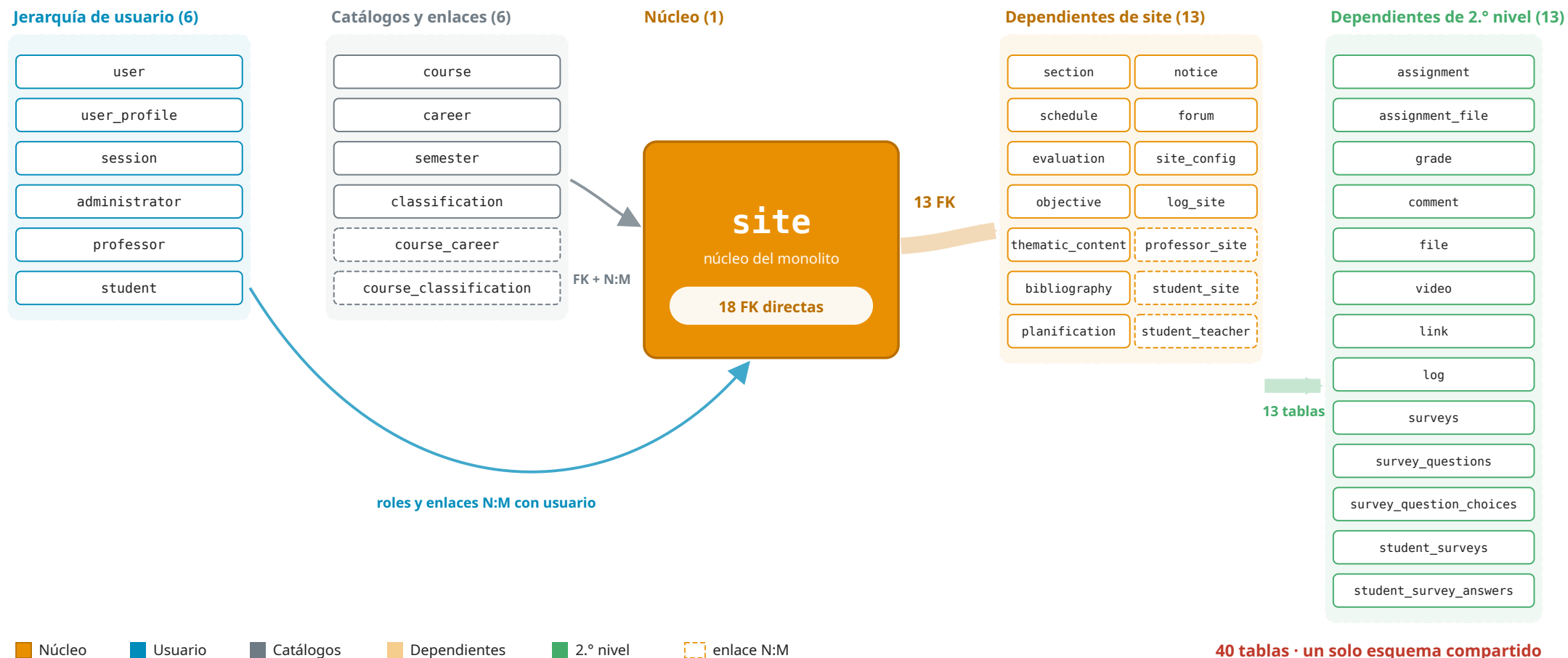
PROBLEMA

El backend monolítico hoy

- Un único proceso concentra **todos los dominios**: autenticación, sitios, evaluaciones, correos, foros...
- Un controlador de **3.629 líneas** gestiona **18 dominios** de negocio
- Un solo esquema de datos: **40 tablas** acopladas referencialmente (siguiente lámina)
- Sin pruebas automatizadas ni observabilidad
- Despliegue manual, infraestructura *on-premise*

PROBLEMA

Un solo esquema: 40 tablas, acoplamiento total



...pero el problema de fondo es humano

Deuda técnica

Seis años de cambios sin proceso sistemático: código densificado, dominios acoplados, cero validación automatizada.

Conocimiento tácito

Colaboradores temporales de 1-2 semestres, sin documentación exhaustiva ni transferencia formal: el saber se pierde en cada rotación.

Carga doble: negocio + DevOps

El colaborador no solo debe aprender la compleja lógica de negocio: también debe hacer DevOps para operar el servicio —sin un equipo permanente de operaciones que lo respalde.

“

Los colaboradores que mantienen PortalAsig son normalmente temporales: permanecen entre uno y dos semestres. Cada rotación pierde parte del conocimiento, dificulta el mantenimiento y la mejora, y alarga la curva de aprendizaje de los nuevos mantenedores.

— El problema que motiva este trabajo

Objetivo general y específicos

Objetivo general

Diseñar e implementar una arquitectura de microservicios para el backend de PortalAsig, adaptar el frontend heredado como cliente de esa arquitectura distribuida e incorporar técnicas sistemáticas de mantenimiento asistidas por inteligencia artificial.

Objetivos específicos

- Documentar el estado actual y caracterizar sus dominios
- Diseñar la arquitectura de microservicios desacoplada
- Adaptar el frontend heredado como cliente distribuido
- Implementar aseguramiento sistemático de calidad
- Diseñar e implementar PAIMA: ciclo MAPE-K con IA
- Instrumentar observabilidad distribuida
- Evaluar el ecosistema con pruebas y experimentos

Tres líneas de transformación

① Backend → Microservicios

Dominios acotados UAA, Site y Notify; cada uno con su base de datos, su versionado y su despliegue autónomo.

② Frontend adaptado

El cliente Vue.js heredado se reestructura para operar contra la arquitectura distribuida con OAuth2/JWT.

③ PAIMA

Servicio de autogestión del mantenimiento: monitorea, diagnostica con IA y propone acciones. El humano aprueba.

Más una cuarta transversal: **técnicas sistemáticas de mantenimiento** (calidad, pruebas y observabilidad) sobre todo el ecosistema.

SECCIÓN

02

Marco conceptual

Las tres ideas que sustentan la solución: microservicios para desacoplar, ingeniería de mantenimiento para sistematizar la calidad, y computación autónoma (MAPE-K) para operar con ayuda de la IA.

De monolito a microservicios

Arquitectura monolítica

- ✗ Un proceso y una base de datos para todo
- ✗ Fallos sin aislamiento entre dominios
- ✗ Despliegue acoplado: todo o nada
- ✗ Un solo esquema acoplado compartido por 18 dominios

VS

Microservicios

- ✓ Servicios autónomos por dominio de negocio
- ✓ Aislamiento de fallos entre servicios
- ✓ Despliegue independiente por servicio
- ✓ Esquema de datos propio por servicio, sin FK cruzadas

A cambio, introduce complejidad operativa: más procesos, más bases de datos, monitoreo distribuido. Esa complejidad es la que aborda PAIMA.

Ingeniería de mantenimiento de software

El estándar **ISO/IEC/IEEE 14764** clasifica el mantenimiento en cuatro tipos —y las técnicas modernas lo convierten en un proceso **sistemático, medible y reproducible**:

Correctivo

Reparar defectos detectados en producción (el 500 que devolvía una evaluación mal sembrada).

Adaptativo

Ajustar el sistema a entornos nuevos (Node 12 fuera de soporte, navegadores actuales).

Perfectivo

Mejorar atributos sin cambiar funcionalidad (rendimiento, legibilidad, deuda técnica).

Preventivo

Evitar fallos futuros: pruebas automatizadas, cobertura exigida, esquema versionado, observabilidad.

Gran parte de este trabajo es mantenimiento preventivo sistematizado: la validación deja de depender del conocimiento tácito y la exige el pipeline en cada cambio.

Computación autónoma: el ciclo MAPE-K

M • Monitor

Recolecta métricas, logs y eventos del entorno gestionado.

A • Analyze

Interpreta síntomas, detecta anomalías y las clasifica.

P • Plan

Decide la acción de remediación a partir del diagnóstico.

E • Execute

Aplica la acción sobre el sistema y verifica el resultado.

K • Knowledge

Base de conocimiento que retroalimenta cada fase del ciclo.

+ IA

Línea de investigación reciente: delegar Analyze y Plan en modelos de lenguaje. PAIMA implementa esta idea.

Stack tecnológico seleccionado

Categoría	Tecnologías	Rol en el ecosistema
Framework de microservicios	Spring Boot (Java)	Servicios UAA, Site, Notify y PAIMA
Persistencia	MySQL + Flyway	Una BD por servicio, esquema versionado
Calidad continua	Checkstyle · Testcontainers · JaCoCo · GitHub Actions	Validación automática en cada pull request
Observabilidad	Prometheus · Grafana · Loki	Métricas, tableros y agregación de logs
Pruebas	JUnit · Cypress · k6	Integración, end-to-end y carga
Inteligencia artificial	LLM local o remoto, configurable	Diagnóstico y planificación en PAIMA (con fallback determinista)
Infraestructura	Docker Compose · Nginx · Config Server	Orquestación local, proxy y configuración centralizada

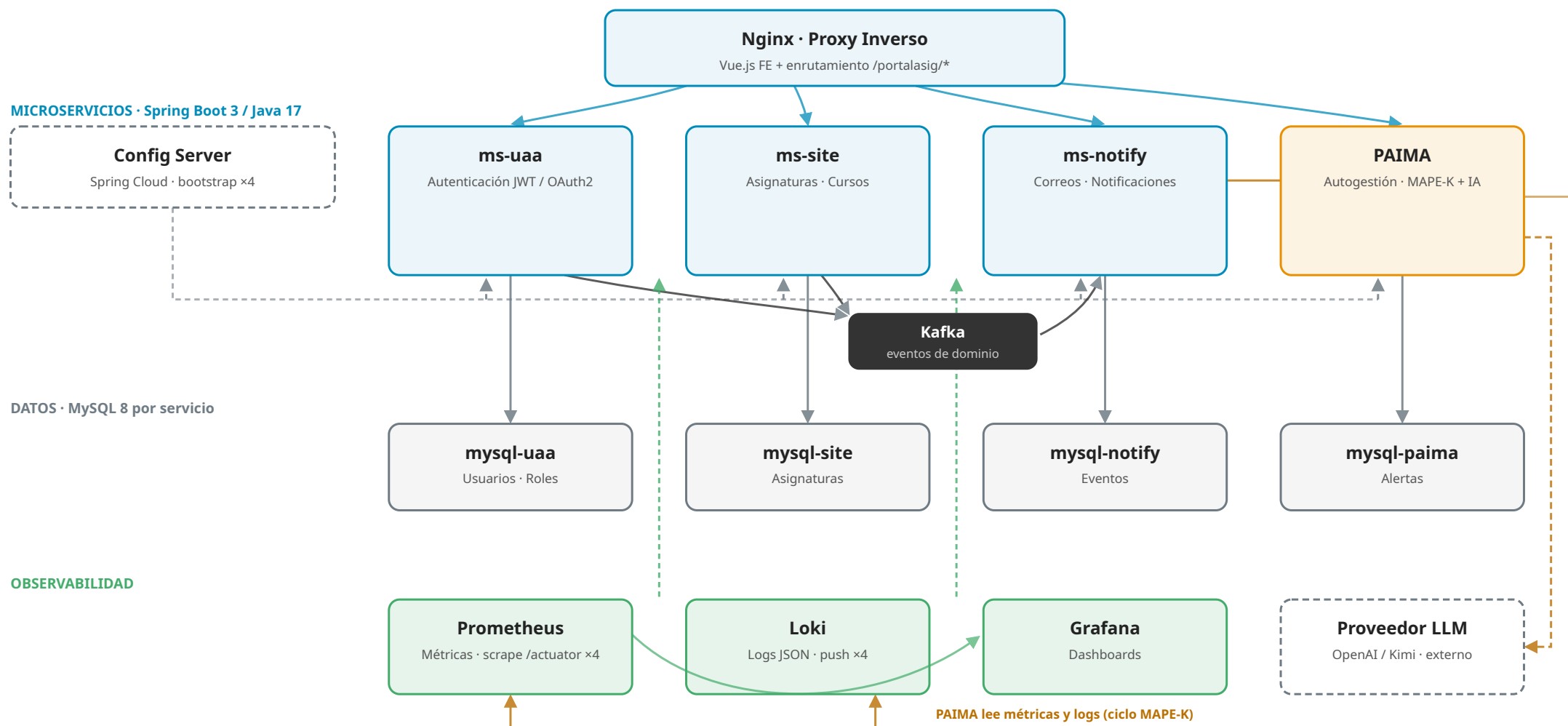
SECCIÓN

03

Solución

Cómo quedó construido: dominios acotados con database per service, calidad exigida automáticamente por pipeline, frontend heredado adaptado, OAuth2/JWT de extremo a extremo y PAIMA como capa de autogestión.

Arquitectura general del ecosistema



Descomposición en dominios acotados

UAA · Identidad y acceso

Usuarios, roles y autenticación OAuth2/JWT con emisión y refresco de tokens.

Site · Núcleo académico

Cursos, semestres, sitios académicos, secciones, horarios y evaluaciones.

Notify · Notificaciones

Renderizado y despacho de correos electrónicos del ecosistema.

PAIMA · Autogestión

Ciclo MAPE-K con IA: vigilancia, diagnóstico y remediación asistida.

Transversales:

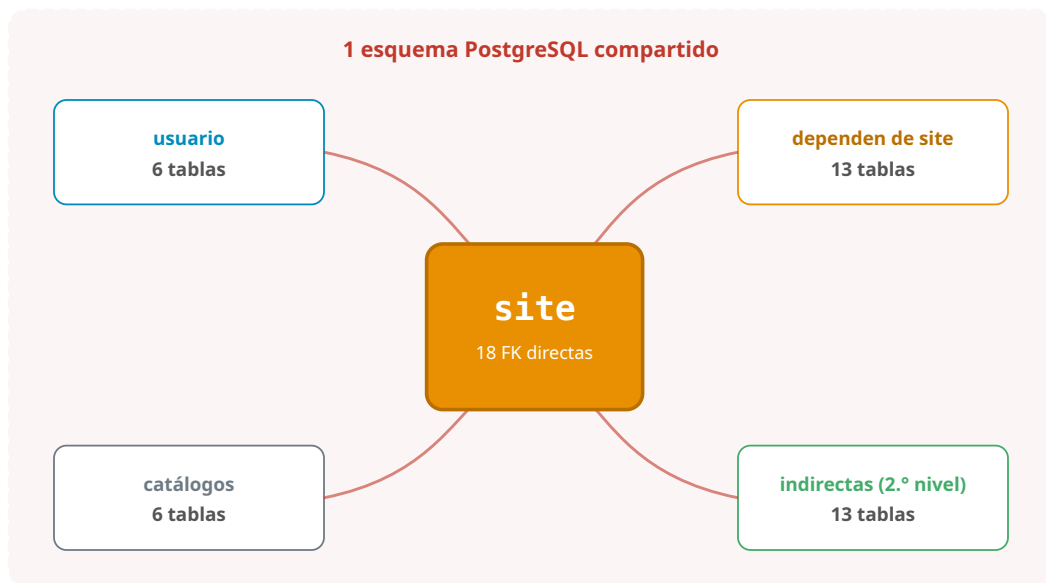
Commons · librería compartida

Config Server · configuración centralizada

SOLUCIÓN

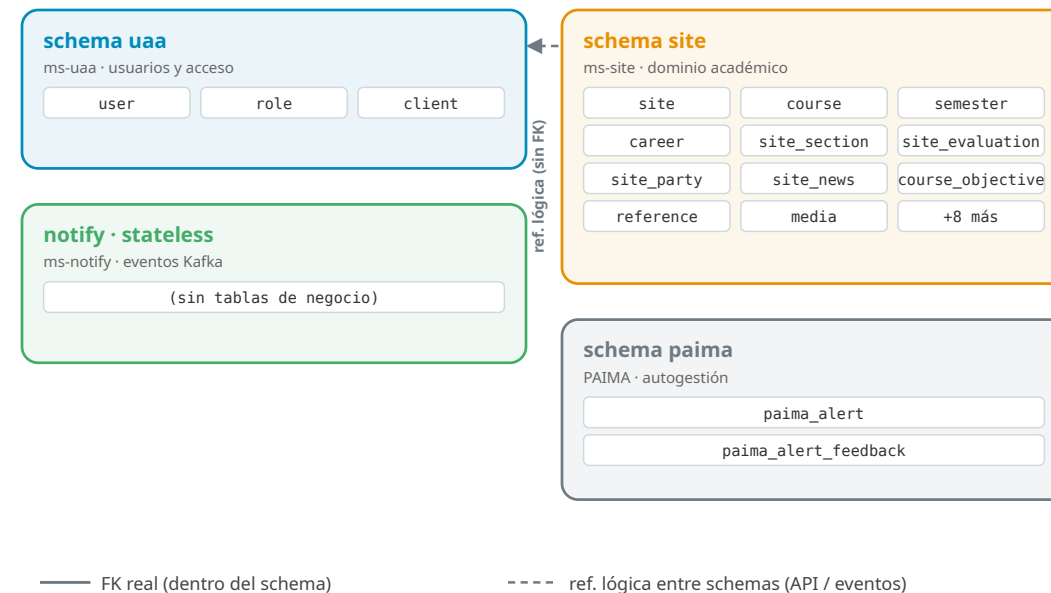
Database per service

Cada dominio **posee y versiona** su propio esquema MySQL: se elimina el acoplamiento referencial del modelo de datos anterior.



40 tablas · acoplamiento referencial total

Antes · Esquema único, dominios acoplados



4 esquemas aislados · cero FK cruzada

Después · Una base de datos por servicio

La calidad deja de depender del conocimiento tácito

- Pipeline en **cada pull request**: Checkstyle → pruebas de integración con **Testcontainers** (BD real) → cobertura **JaCoCo**
- **Branch protection**: si un check falla, el merge queda bloqueado
- Validación **sistemática y reproducible**, no manual ni dependiente de memoria

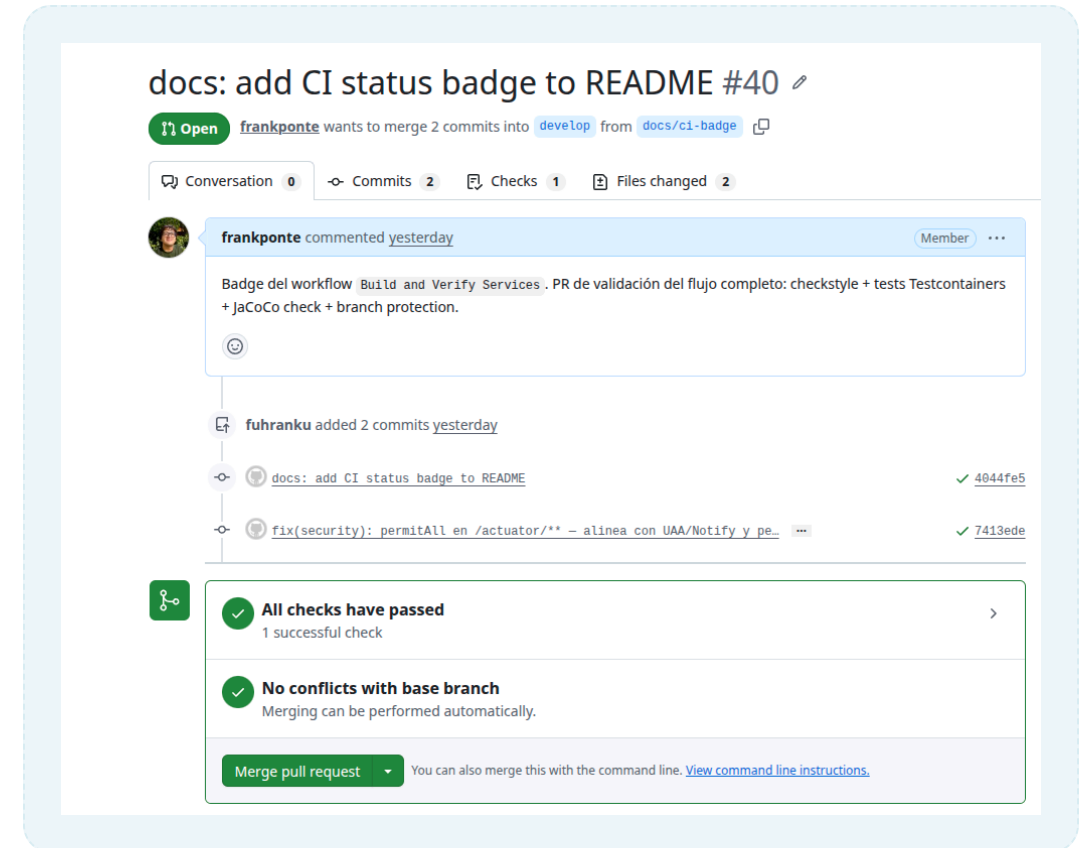


Fig. 3 · El PR no se fusiona hasta que el pipeline pasa.

Esquema versionado con Flyway

```
V1.0.0.00_init.sql
1 CREATE TABLE course
2 (
3   course_id INT NOT NULL AUTO_INCREMENT COMMENT 'primary key, auto increment',
4   code VARCHAR(128) NOT NULL COMMENT 'Course code. Serves as a identifier',
5   name VARCHAR(128) NOT NULL COMMENT 'Course name',
6   credit_units SMALLINT UNSIGNED NOT NULL COMMENT 'Number of credit units',
7   type VARCHAR(128) NOT NULL COMMENT 'Course type',
8   course_level VARCHAR(32) NOT NULL COMMENT 'Course level where is required: Semester 1st, 2nd, etc...',
9   requirements VARCHAR(255) COMMENT 'Course requirements',
10  created_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'date and time this row was created',
11  updated_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT 'date and time this row was last updated',
12  PRIMARY KEY (course_id),
13  UNIQUE KEY (code),
14  KEY course_idx1 (type)
15 ) COMMENT 'A course that is offered by college';
16
17 CREATE TABLE course_semester_link
18 (
19   course_semester_link_id INT NOT NULL AUTO_INCREMENT COMMENT 'primary key, auto increment',
20   course_id INT NOT NULL COMMENT 'Course ID',
21   semester_id INT NOT NULL COMMENT 'Semester ID',
22   created_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'date and time this row was created',
23   updated_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT 'date and time this row was last updated',
24   PRIMARY KEY (course_semester_link_id)
25 ) COMMENT 'A course that is offered in a semester';
26
27 CREATE TABLE course_career_link
28 (
29   course_career_link_id INT NOT NULL AUTO_INCREMENT COMMENT 'primary key, auto increment',
30   course_id INT NOT NULL COMMENT 'Course ID',
31   career_id INT NOT NULL COMMENT 'Career ID',
32   created_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'date and time this row was created',
33   updated_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT 'date and time this row was last updated',
34   PRIMARY KEY (course_career_link_id)
35 ) COMMENT 'Join table between course and career';
36
37 CREATE TABLE career
38 (
39   career_id INT NOT NULL AUTO_INCREMENT COMMENT 'Career ID',
```

- Migraciones SQL incrementales:
`V{versión}__{descripción}.sql`
- El esquema se construye **progresivamente y de forma reproducible**
- Un colaborador nuevo levanta la base de datos **sin conocimiento tácito**

Fig. 4 · Extracto de la migración inicial del servicio Site.

SOLUCIÓN

El frontend heredado se adapta, no se reescribe

- Store **Vuex reestructurado en módulos por dominio**, alineados con los microservicios
- Guards de navegación con **7 niveles de permisos** (estudiante → administrador)
- Múltiples instancias **Axios** que reflejan la topología: UAA, Site, PAIMA
- Sesión en dos capas: **client_credentials** (aplicación) + login de usuario con **refresh token** en cookies

Se sigue el precedente del TEG de 2020: evolución controlada en lugar de reescritura completa.

Autenticación OAuth2/JWT de extremo a extremo

- Se reemplaza la autenticación propietaria por **OAuth2/JWT estándar**
- Token de aplicación (client_credentials) + token de usuario con refresh
- CORS configurado por microservicio; flujo **validado E2E con Cypress**

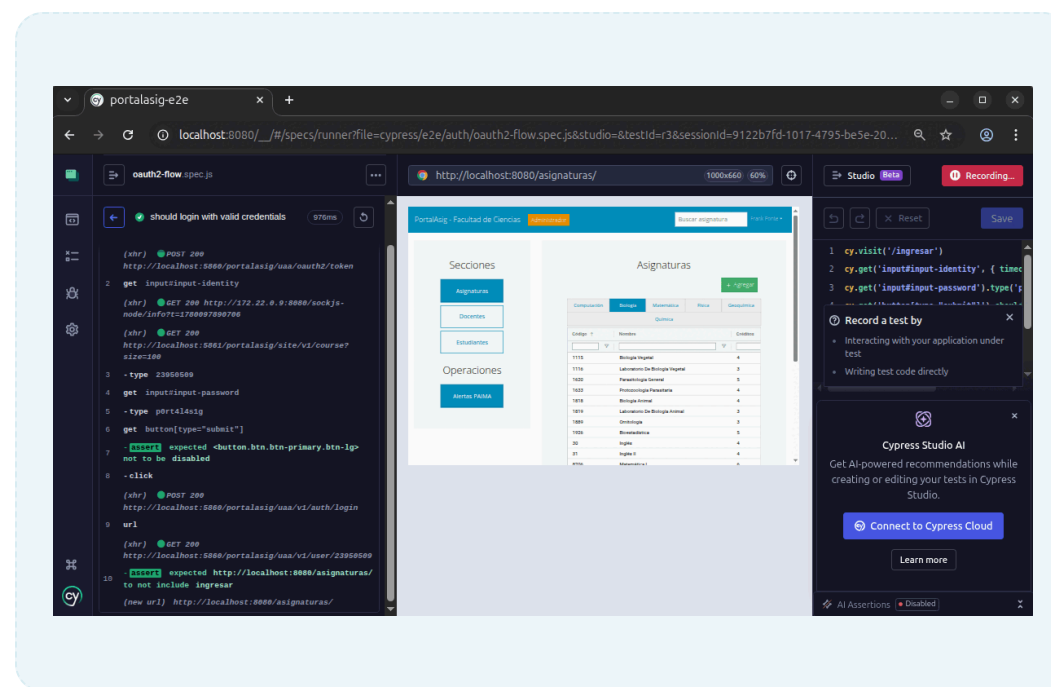
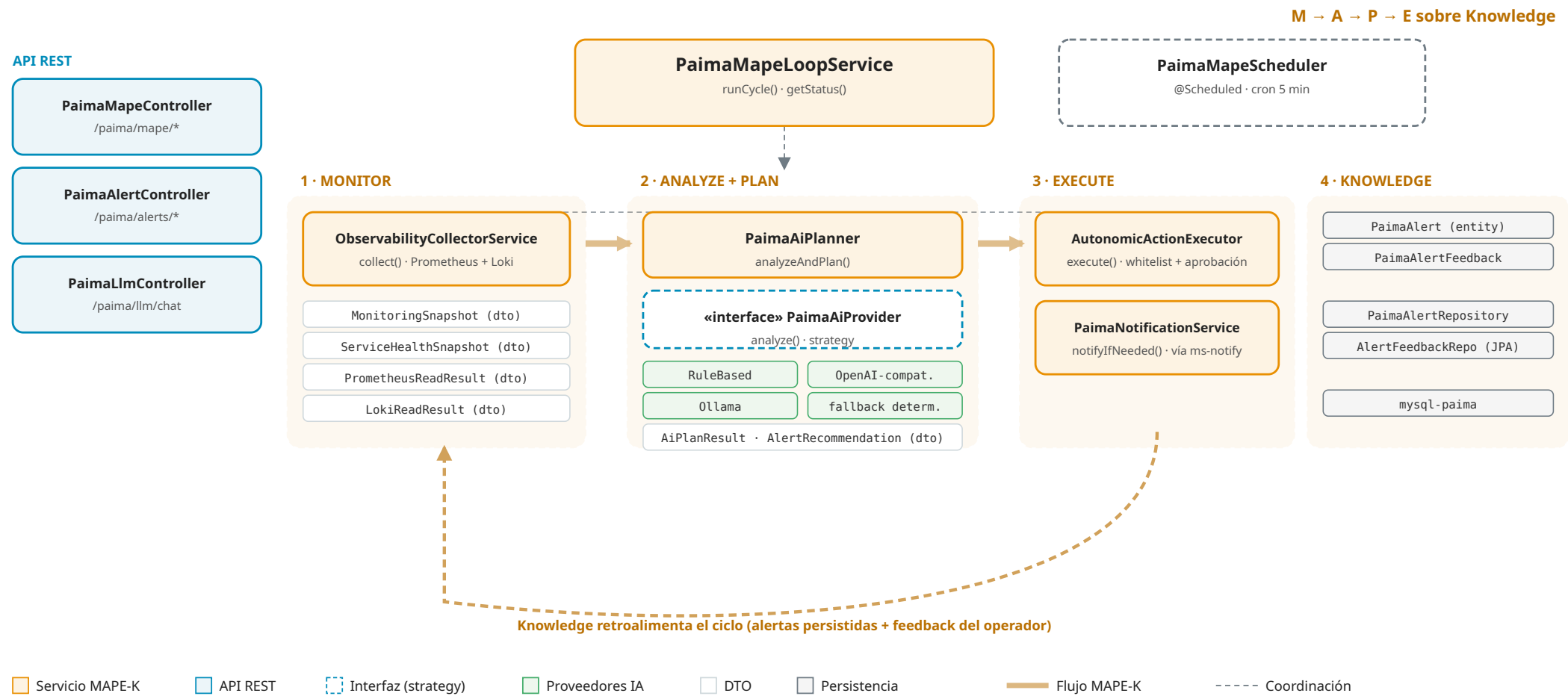


Fig. 5 • Flujo de autenticación validado de extremo a extremo.

PAIMA: PortalAsig Intelligent MAPE-K

- Se construye **sobre la observabilidad ya instalada**: Prometheus, Loki, Grafana
- Las fases **Analyze y Plan se delegan a un LLM** — proveedor configurable con **fallback determinista**
- **Human-in-the-loop**: el operador evalúa y aprueba cada acción antes de ejecutarse
- Su interior es **el ciclo MAPE-K implementado en clases** — siguiente lámina
- **portalasig-cli**: script propio de operación para interactuar con todos los servicios desde la terminal

PAIMA por dentro: el ciclo MAPE-K en clases



Operación pensada para quien llega “en frío”

- Salud del ecosistema **en un vistazo**: servicios e incidentes activos
- El panel cubre **salud e incidentes**; las métricas y dashboards detallados viven en **Grafana**
- Reduce la curva de entrada de un **nuevo mantenedor**
- Documentación API **autogenerada** (Swagger/OpenAPI desde el propio código)
- **portalsig-pi**: repositorio de conocimiento del proyecto que cualquier agente de IA usa para hacer onboarding y asistir con eficacia a un nuevo colaborador
- El panel real, capturado durante la simulación de caos — siguiente lámina

El panel de PAIMA en operación real

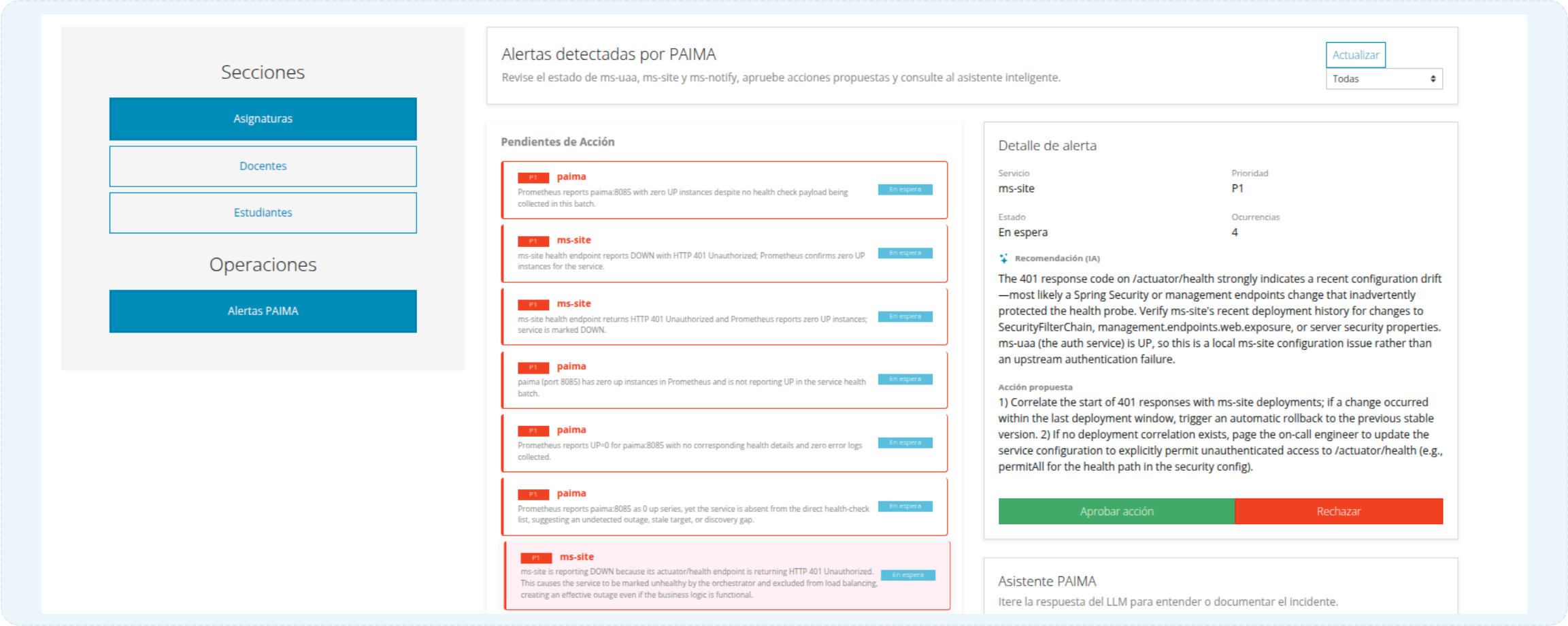


Fig. 7 · Panel de gestión de PAIMA durante la simulación de caos: salud de servicios, incidentes P1 y la recomendación de la IA.

IA que propone, humano que decide

- El operador **conversa con el asistente** para entender la incidencia detectada
- La IA **clasifica, diagnostica y propone** la acción de remediación dentro del ciclo MAPE
- El chat es **consultivo**: no altera el diagnóstico ni el plan — la decisión se materializa solo al **aprobar o rechazar**
- Nada se ejecuta sin **aprobación explícita** del operador

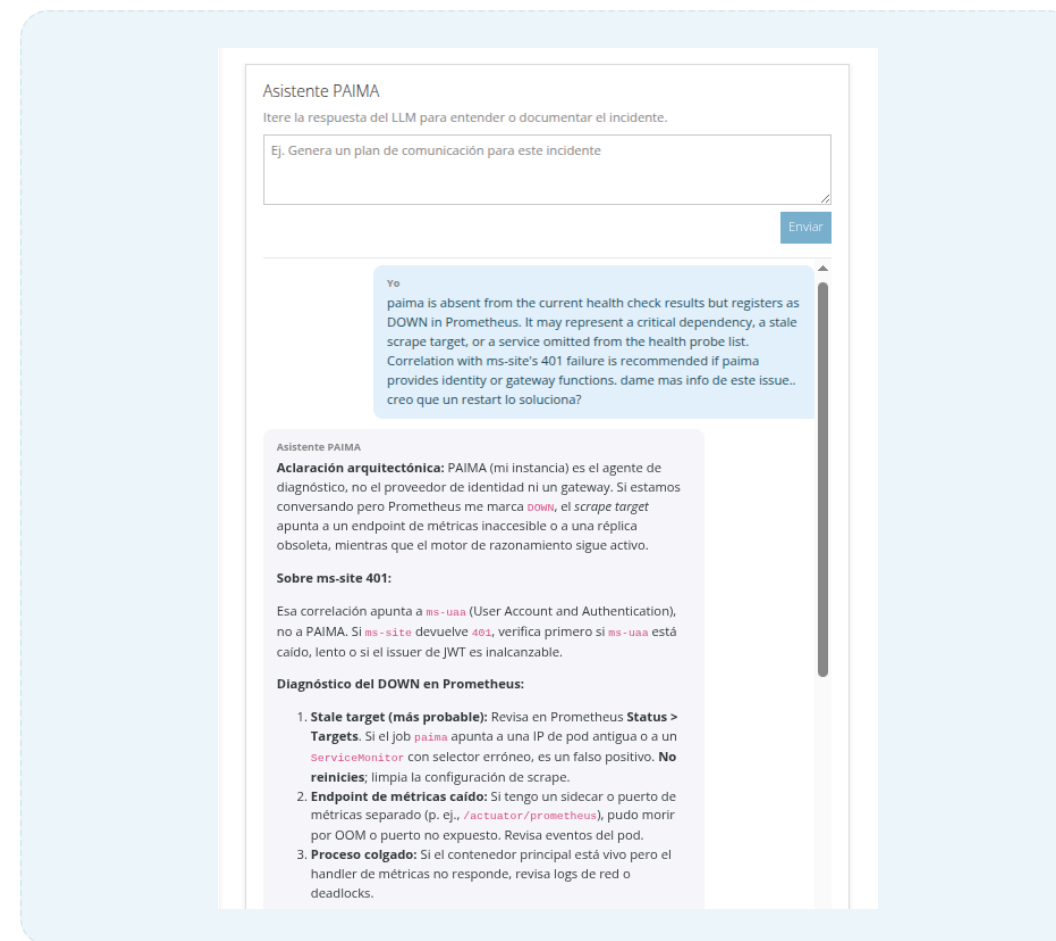


Fig. 8 · Asistente conversacional de diagnóstico.

SECCIÓN

04

Validación y resultados

Cuatro dimensiones de evaluación con evidencia real: cobertura medida y exigida, flujos E2E con Cypress, benchmark término a término frente al monolito y simulación de caos con recuperación asistida.

Estrategia de pruebas en cuatro dimensiones

Dimensión	Qué mide	Herramientas	Criterio
Integración	Cobertura de líneas y ramas	JaCoCo + GitHub Actions	Umbral 70% líneas / 60% ramas
End-to-end	Flujos completos de usuario	Cypress	Suite estable en verde
Rendimiento	Latencia media/p95, throughput, errores	k6 + Prometheus	Comparativo frente al monolito
Resiliencia	Detección y recuperación de fallos	Simulación de caos + PAIMA	Recuperación asistida validada

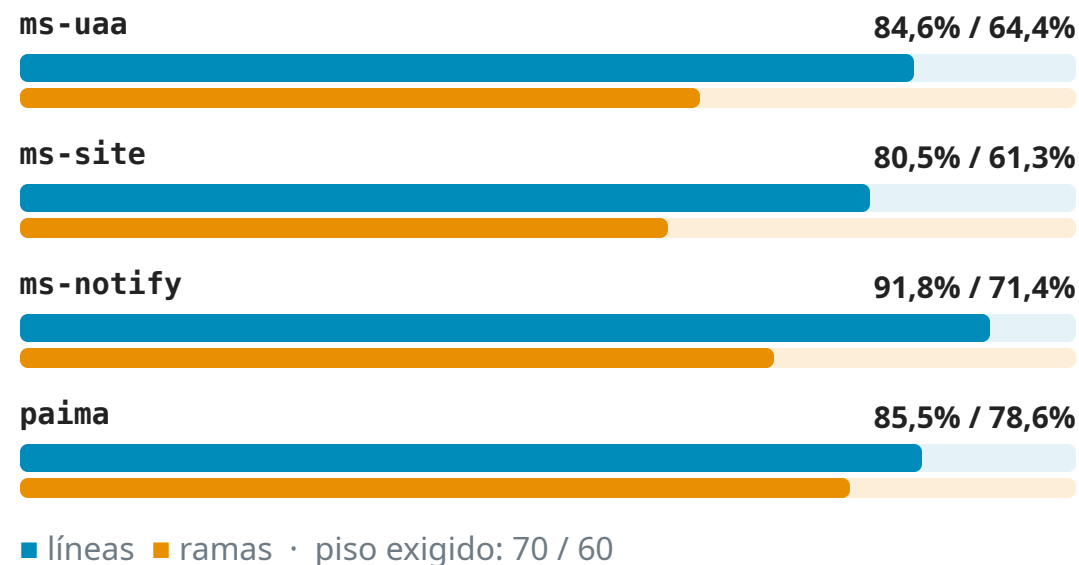
Entorno: Docker Compose local, datos anonimizados, diseño experimental reproducible.

Cobertura medida y exigida automáticamente

Cómo se mide y se exige

- **JaCoCo** mide líneas y ramas en cada build del pipeline
- Pruebas de integración con **Testcontainers + MySQL 8 real** — no H2 ni mocks
- **Piso 70/60**: JaCoCo exige que al menos el **70% de las líneas** y el **60% de las ramas** (los dos caminos de cada *if/else*) de cada servicio estén cubiertas por pruebas. Si un cambio baja de ese piso, el build falla y GitHub **bloquea el merge**
- Checkstyle + **branch protection** con check requerido en los 4 repos

Cobertura por servicio (líneas / ramas)



De **cero** (el monolito no tenía ni una prueba) a cobertura **medida y exigida** en cada cambio.

Flujos de usuario validados con Cypress

12

specs automatizados

52

casos de prueba

1m16s

corrida completa en verde (Docker Compose)

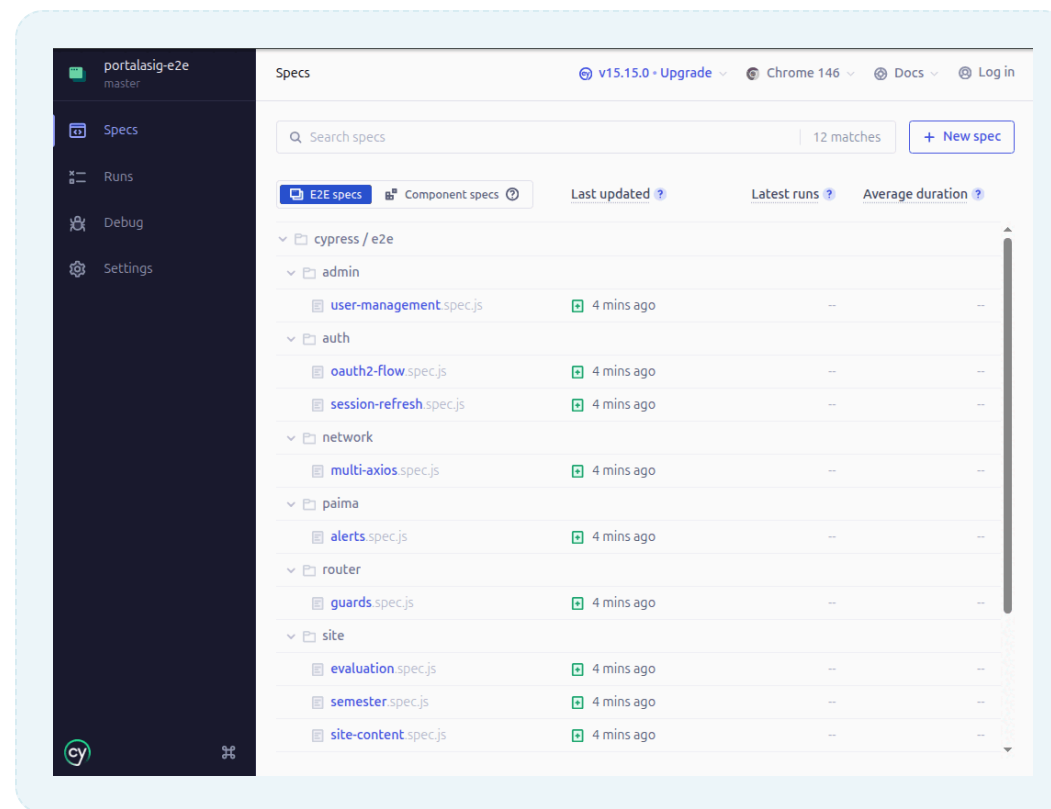


Fig. 9 · Ejecución completa de la suite E2E.

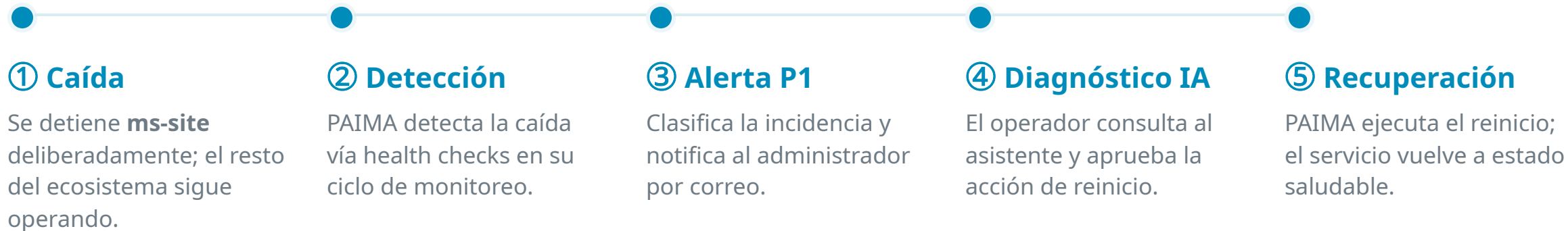
Benchmarking con k6

50 usuarios virtuales, 3 escenarios con endpoints equivalentes, idénticos recursos y los mismos datos en ambas arquitecturas:

Escenario	Monolito legacy media / p95	Microservicios media / p95	Lectura
Lectura · cursos	262,9 / 304,9 ms 181 req/s	136,7 / 258,4 ms 348 req/s · 0% error	MS 1,9× más rápido, casi 2× throughput
Escritura · crear curso	90,4 / 117,0 ms error 1,39%	85,9 / 119,0 ms 0% error	Empate; MS robusto (idempotente) vs 500 del legacy
Auth · login + perfil	37,2 / 80,0 ms	168,9 / 246,2 ms	bcrypt deliberado (MD5 en legacy); el costo solo lo paga UAA

0 fallas de plataforma en 390 mil peticiones · la latencia de login se concentra en UAA y se escala solo ese servicio · entorno controlado local, no producción.

Simulación de caos: caída de un servicio



La evidencia del experimento

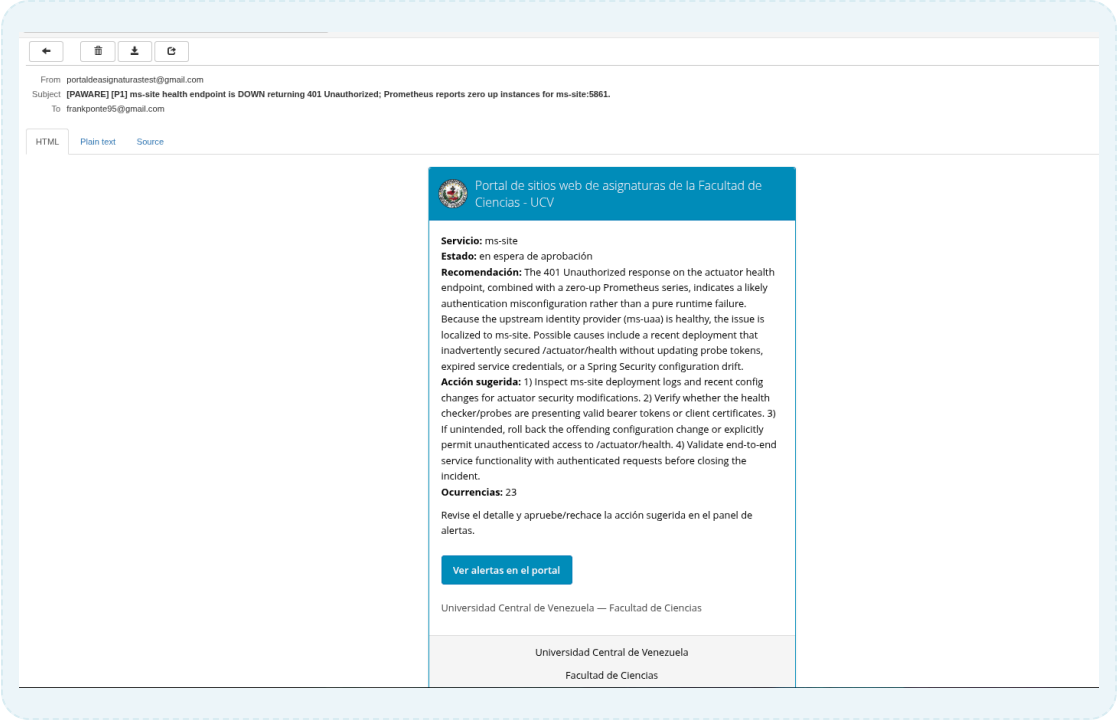


Fig. 11 · Alerta P1 recibida por correo electrónico.

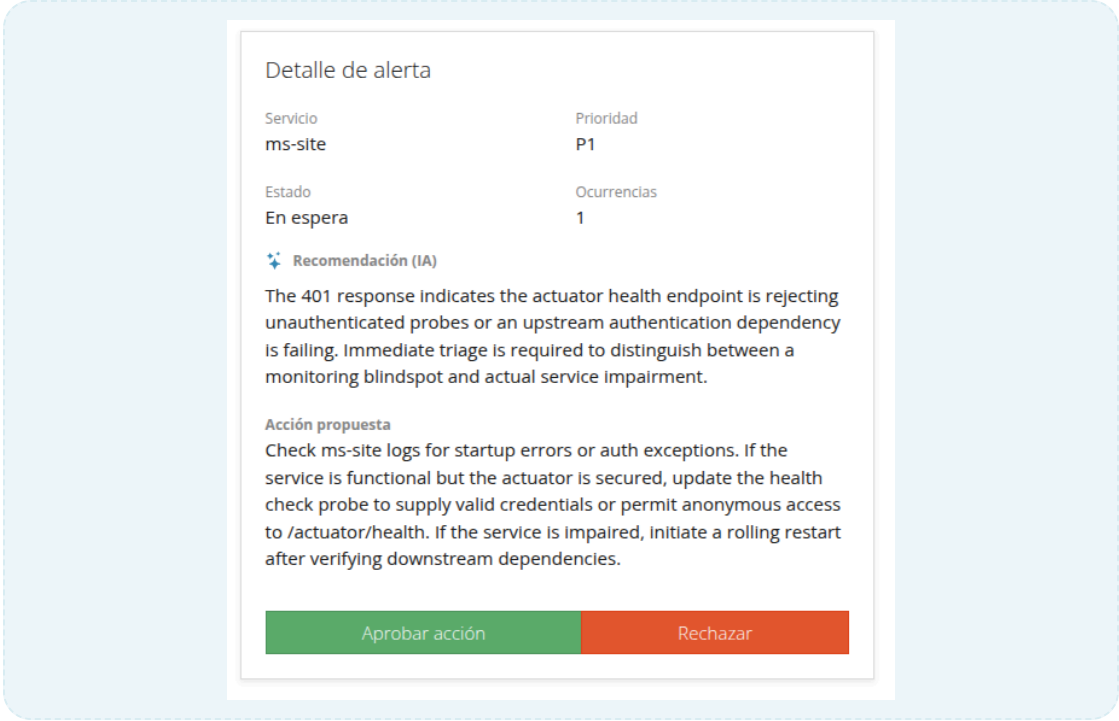


Fig. 12 · Detalle de la incidencia en el panel de PAIMA.

Análisis comparativo global

Dimensión	Monolito legacy	Ecosistema de microservicios
Cobertura de código	Inexistente	80,5–91,8% medida y exigida por PR
Validación de cambios	Manual, alto riesgo	Pipeline automático con merge bloqueado
Resiliencia	Un fallo compromete todo	Aislamiento + recuperación asistida
Observabilidad	Nula	Prometheus, Grafana y Loki centralizados
Mantenimiento	Conocimiento tácito	Sistemático y reproducible

Lo que quedó demostrado

- ✓ El ecosistema **aísla fallos**: la caída de un servicio no detiene al sistema
- ✓ Rinde mejor: **1,9×** en lectura concurrente y **0% de errores** en 390 mil peticiones
- ✓ La calidad se **mide y se exige** automáticamente en cada cambio
- ✓ PAIMA **reduce la carga cognitiva**: detecta, diagnostica y propone; el humano solo aprueba
- ✓ Un colaborador nuevo puede **operar el sistema sin conocimiento tácito**

SECCIÓN

05

Conclusiones

Tres transformaciones demostradas con números, las limitantes declaradas con honestidad y las líneas de evolución que el trabajo deja abiertas.

Tres transformaciones interdependientes

- ✓ **Arquitectónica:** backend migrado a dominios acotados (UAA, Site, Notify), cada uno con BD propia y despliegue autónomo; frontend adaptado como cliente OAuth2 distribuido
- ✓ **Metodológica:** validación automatizada en cada cambio — cobertura exigida por pipeline, E2E verde, esquemas versionados
- ✓ **Operativa:** PAIMA ejecuta el ciclo MAPE-K con IA y fue validado en simulación de caos

LIMITACIONES

Limitantes y aspectos a mejorar

- ✗ El ciclo de monitoreo de PAIMA (**5 min**) acota el tiempo de detección de una incidencia
- ✗ La calidad del diagnóstico **depende del proveedor de IA**: remoto razona mejor; local/fallback es más simple
- ✗ El benchmark se ejecutó en **ambiente local controlado** (idénticos recursos para ambas arquitecturas): los valores absolutos no representan producción
- ✗ Frontend sobre **Vue 2** (fin de vida)

Líneas de evolución identificadas

- **API Gateway:** routing y autenticación centralizados en el borde
- **Soberanía tecnológica:** modelos de lenguaje locales y especializados en operaciones
- **PAIMA** hacia un agente semi-integrado de desarrollo y, a largo plazo, una arquitectura multi-agente
- **Frontend:** migración a Vue 3 y cobertura de pruebas en el cliente
- Migración de **datos históricos** de producción y **trazabilidad distribuida** (trazas con trace-id, OpenTelemetry)

“

Un sistema desacoplado, observable y autogestionado, que puede ser mantenido por colaboradores académicos temporales sin depender del conocimiento tácito.

— El aporte central de este trabajo

¡Gracias!

Quedo atento a las preguntas del jurado

Br. Frank Ángel Ponte Delgado

Tutor: Prof. Walter Hernández

PortalAsig · Facultad de Ciencias · UCV